# Laboratorul 5

1. **Metodele abstracte pot fi final**
   - **True**
   - **False**
2. **O clasa poate sa mosteneasca mai multe clase**
   - **True**
   - **False**
3. **Cum sunt campurile unei interfețe?**
   - **static, final, public**
   - **final, protected**
   - **public**
   - **static, final**
4. **Interfețele nu pot implementa alte interfețe**
   - **True**
   - **False**
5. **Dacă Feline extinde clasa abstracta Animal și Lion extinde Feline, atunci care instantiere este corecta?**
   - **Feline feline = new Animal();**
   - **Animal animal = new Feline();**
   - **Lion lion = new Animal();**
   - **Lion lion = new Feline();**
6. **Pot avea clasele abstracte constructori?**
   - **True**
   - **False**
7. **Care linii sunt incorecte? (1-5)**

```
interface ITest {
  protected int x = 10;
  int y;
  int z = 20;
  abstract void foo();
  final int f(int x);
}
```

   - **1, 2, 5**
   - **1, 2, 3, 4, 5**
   - **2, 3, 4**
   - **4, 5**

8. **Clasa A implementează interfața B cu metoda statică f. Cum o putem apela?**
    - **B.f();**
    - **f();**

# Laboratorul 6

9. **Are acces o clasa internă la membrii clasei externe?**
    - **True**
    - **False**
10. **O clasa internă privată are constructori publici**
    - **True**
    - **False**
11. **Clasele anonime nu au nume**
    - **True**
    - **False**
12. **Clasele interne anonime pot avea constructori diferiti de cel default**
    - **True**
    - **False**
13. **Ce nu poate sa faca o clasa anonima?**
    - **Sa implementeze o interfata**
    - **Sa implementeze mai multe interfete**
    - **Sa mostenească o clasa**
    - **Sa acceseze membrii clasei externe**
14. **Numai variabilele de tip final declarate in metoda pot fi accesate de clasa interna**
    - **True**
    - **False**
15. **Pentru a instantia o clasa interna non statica avem nevoie de un obiect al clasei externe**
    - **True**
    - **False**
16. **Pentru a instantia o clasa interna statica avem nevoie de un obiect al clasei externe**
    - **True**
    - **False**
17. **O clasa interna statica are acces la campurile nestatice ale clasei externe**
    - **True**
    - **False**
18. **Ce se afiseaza?**

- 12
- 2
- Eroare compilare, nu putem apela explicit main din Inner
- Eroare compilare, nu putem avea metoda main intr-o clasa internă

19. Cum instanțiem o clasa interna anonima de tipul Food (clasă abstractă ce conține metoda public abstract String eat();)
    - Food f = new Food()
    - System.out.println(new Food(public String eat(){return “eat pizza”;}));
    - System.out.println(new Food{String eat(){return “eat pizza”;}});
    - System.out.println(new Food(){public String eat(){return “”;}});
20. Care afirmație despre clasele interne statice este adevărată?
    - Trebuie să avem o referință la clasa exterioara pentru a o putea instanția
    - Din clasa interna nu avem acces la membrii non-statici ai clasei exterioare
    - variabilele și metodele ei trebuie să fie statice
    - o clasă internă statică nu poate avea specificatorul de acces private
21. Care din următoarele afirmații este adevărată despre clasele interne statice în Java:
    - Este nevoie de o referință la clasa externă pentru a instanția clasa intern
    - Nu are acces la membrii non-statici ai clasei externe
    - Atributele și metodele conținute de aceasta trebuie să fie statice
    - Trebuie neapărat să moștenească clasa externă

# Laboratorul 7

22. Ce nu include semnatura unei functii?
    - nume metoda
    - numarul + tipul parametrilor
    - Exceptii
    - valoarea de return
23. Pot sa fac overloading daca modific numai tipul returnat
    - True
    - False
24. Overloadingul este un polimorfism dinamic?
    - True
    - False
25. Overriding - pot schimba tipul returnat
    - True
    - False
26. Nu pot suprascrie constructorii

- **True**
- **False**

27. **La overriding pot pune un specificator de acces mai restrictiv**
    - **True**
    - **False**
28. **O metoda final poate fi suprascrisa**
    - **True**
    - **False**
29. **Care este diferenta dintre un tip primitiv si un wrapper?**
    - **Wrapper-ul poate fi si NULL**
    - **nu pot avea campuri de tip primitiv intr-o clasa**
    - **Niciuna?**
    - **numai obiectele de tip Wrapper sunt imutabile**
30. **Ce fel de design pattern e Visitor?**
    - **Structural**
    - **Behavioral**
    - **Creational**
    - **este design pattern???**
31. **E ok sa folosim instanceof**
    - **True**
    - **False**
32. **E recomandat sa folosim Visitor cand avem multe tipuri noi Visitable**
    - **True**
    - **False**
33. **Atat single dispatch cat si double dispatch sunt la compile time**
    - **True**
    - **False**
34. **Ce dezavantaj are Visitor?**
    - **separam logica fiecarei operatii**
    - **putem gestiona tipuri diferite de obiecte**
    - **usureaza adaugarea de noi operatii**
    - **depinde de stabilitatea structurii**

# Laboratorul 8

35. **Ce design pattern am folosi, dacă în aplicație avem foarte multe notificări între componente?**
    - **Visitor**
    - **Command**
    - **Observer**
    - **Factory**

36. **Cu ce design pattern am putea modela comportamentul de onClick → doSomething?**
    - **Visitor**
    - **Singleton**
    - **Factory**
    - **Observer**
37. **Avem o colecție de date și dorim să facem o statistică relevantă în funcție de un criteriu. Ce design pattern folosim?**
    - **Visitor**
    - **Command**
    - **Factory**
    - **Strategy**
38. **Care dintre următoarele patternuri permite crearea de obiecte fără a expune logica lor clientilor?**
    - **Observer**
    - **Visitor**
    - **Factory**
    - **Singleton**
39. **Ce pattern se foloseste daca vrem adaugarea de noi functionalitati in viitor, fara modificarea structurilor existente?**
    - **Factory**
    - **Visitor**
    - **Pizza**
    - **Singleton**
40. **Care pattern implică ascunderea vizibilității contructorilor unei clase?**
    - **Singleton**
    - **Visitor**
    - **Observer**
    - **Command**
41. **Ce fel de design pattern este Singleton?**
    - **Creational**
    - **Structural**
    - **Behavioural**
    - **Architectural**
42. **Ce fel de design pattern este Visitor?**
    - **Creational**
    - **Structural**
    - **Behavioural**
    - **Architectural**
43. **Ce fel de design pattern este Factory?**
    - **Creational**

- **Structural**
- **Behavioural**
- **Architectural**

44. **Ce fel de design pattern este Observer?**
    - **Creational**
    - **Structural**
    - **Behavioural**
    - **Architectural**
45. **Ce fel de design pattern este Strategy?**
    - **Creational**
    - **Structural**
    - **Behavioural**
    - **Architectural**

**Raspunsuri**

1. **False**

   Metode final => nu mai pot fi suprascrise
   Metode abstract => nu oferă o  implementare

   Metode final + abstract => nu avem o implementare și nici nu putem sa suprascriem metoda ca sa o implementăm in clasele copil
   (**eroare compilatorul nu va ști ce implementare să asocieze metodei**)

2. **False**

   In Java o clasa **poate extinde mai multe interfețe, dar nu poate mosteni decat o singura clasa** (altfel, problema diamantului: dacă avem metode cu aceleași nume in clasele parinte atunci ce metode apelam?)

3. **static, final, public**
   - campurile și metodele unei interfețe sunt public by default. Interfata in sine e package-private dacă nu o facem explicit de tip public.
   - toate campurile unei interfețe sunt constante (static + final):
     - final - nu se mai pot modifica (+trebuie inițializate)
     - static - specifice interfetei

4. **True**

   O interfata poate **extinde** mai multe interfețe. O interfata poate extinde o alta interfata, nu o poate implementa.

5. **Animal animal = new Feline();**
   - Animal - clasa abstracta => pica din start variantele:
     - Feline feline = new Animal();
     - Lion lion = new Animal();

     (**nu putem instantia clase abstracte**)
   - Lion lion = new Feline(); nu este corecta
     - **nu orice felina este leu** (putem avea tipuri diferite de feline)
     - Lion este o clasa mai specifică, Feline mai generala => downcasting incorect aici

6. **True**

   Da, clasele abstracte ne spun numai ca **pot** avea metode abstracte (fara implementari), dar ele pot avea constructori, metode statice, etc

7. **1, 2, 5**
   - Linia 1: Java nu permite campuri protected

- Linia 2: campurile unei interfețe sunt final => trebuie inițializate
- Linia 3: all good (avem și inițializare)
- Linia 4: all good (putem pune abstract in fața unei metode)
- Linia 5: nu putem pune final pentru ca altfel nu mai putem oferi o implementare metodei (din același motiv ca la 1)

8. **`B.f();`**
    - Metoda statica (specifica interfetei) -> apel cu nume_interfata.nume_metoda(....)